

Informe final de implementación

Sistema de gestión empresarial EcoTech Solutions

1. Introducción

Este informe presenta la solución implementada para el sistema de gestión empresarial de EcoTech Solutions. La implementación parte del diagrama de clases UML y del informe técnico entregados, y traduce el diseño orientado a objetos a Python.

La solución se organizó en dos archivos principales:

- `ecotech.py`: modelo de dominio, reglas de validación, persistencia SQLite y operaciones CRUD.
- `interfaz.py`: interfaz gráfica de usuario desarrollada con Tkinter.

Esta separación permite que la lógica del sistema funcione independientemente de la interfaz visual y facilita las pruebas, el mantenimiento y futuras ampliaciones.

2. Objetivos solicitados

La solución debía:

1. Implementar las clases y relaciones definidas en el UML.
2. Aplicar encapsulamiento, herencia y abstracción.
3. Incorporar una base de datos SQLite con operaciones CRUD.
4. Validar los datos y controlar errores mediante excepciones específicas.
5. Proporcionar una interfaz con la que el usuario pueda interactuar.
6. Documentar la correspondencia entre UML, código y decisiones de diseño.
7. Incluir una evaluación crítica de los elementos que podrían requerir ajustes manuales.

Todos estos objetivos fueron incorporados en la solución.

3. Correspondencia entre UML y código

3.1 Usuario

Usuario se implementó como clase abstracta mediante `ABC` y `abstractmethod`. Contiene los datos comunes `id`, `nombre` y `correo`, además del método abstracto `puede_acceder(modulo)`.

La clase utiliza atributos privados con prefijo `_` y propiedades de lectura. Esto evita que el estado interno sea modificado directamente sin pasar por validaciones.

3.2 Empleado

Empleado hereda de Usuario y agrega:

- `direccion`
- `telefono`
- `salario`
- `fecha_inicio`
- `id_departamento`

Sus métodos son `get_salario`, `set_salario`, `registrar_horas` y `obtener_proyectos`.

El salario se convierte a Decimal y se valida para impedir valores negativos. La fecha se valida con formato ISO AAAA-MM-DD.

3.3 Administrador

Administrador hereda de Usuario y contiene los métodos crear_departamento y generar_informe.

El informe técnico identificaba que esta clase no tenía atributos propios. Se mantuvo así porque su responsabilidad es funcional y administrativa, pero se dejó documentado que podría incorporar un atributo como nivel_permisos si el caso real lo necesitara.

3.4 Departamento

Departamento contiene id_dept, nombre_dept y gerente. Mantiene una colección de empleados.

La relación con Empleado se implementa como agregación: el departamento mantiene referencias a empleados, pero no controla su ciclo de vida. Sus métodos son agregar_empleado, remover_empleado y listar_empleados.

3.5 Proyecto

Proyecto contiene id_proyecto, nombre, descripcion, fecha_inicio y estado.

La relación muchos a muchos entre empleados y proyectos se representa mediante listas en memoria y mediante la tabla intermedia empleado_proyecto en SQLite. Sus métodos son asignar_empleado, remover_empleado y calcular_total_horas.

3.6 RegistroTiempo

RegistroTiempo contiene id_registro, fecha, horas y descripcion, además de referencias al empleado y al proyecto relacionado.

El método validar_limite_diario impide registrar más de 24 horas en un día. El método obtener_resumen genera una descripción legible del registro.

3.7 ReporteManager

ReporteManager contiene id_reporte, tipo y fecha_generacion. Implementa generar_reporte_empleado, exportar_pdf y exportar_excel.

Las exportaciones actuales escriben el contenido en un archivo indicado por el usuario. Para generar formatos PDF y Excel reales sería necesario integrar librerías especializadas, como reportlab y openpyxl.

4. Arquitectura de la solución

La aplicación se divide en dos capas principales.

4.1 Capa de dominio y persistencia

Está ubicada en ecotech.py e incluye:

- Clases del modelo UML.
- Reglas de negocio y validación.
- Conexión SQLite.
- Creación del esquema.
- Operaciones CRUD.
- Demostración automática de funcionamiento.

4.2 Capa de presentación

Está ubicada en interfaz.py e incluye:

- Ventana principal de Tkinter.
- Pestañas para cada grupo de información.
- Formularios de entrada.
- Tablas para visualizar los registros.
- Botones para crear, consultar, actualizar y eliminar.
- Mensajes de error sin cerrar la aplicación.

La interfaz utiliza las funciones públicas del backend y no duplica la lógica de persistencia.

5. Base de datos SQLite

La función inicializar_bd() crea la base de datos y activa las claves foráneas mediante:

```
PRAGMA foreign_keys = ON
```

El esquema contiene las siguientes tablas:

- usuario
- empleado
- administrador
- departamento
- proyecto
- registro_tiempo
- empleado_proyecto

Las tablas utilizan claves primarias y foráneas para mantener la integridad de las relaciones. La tabla empleado_proyecto representa la asociación muchos a muchos entre empleados y proyectos.

6. Operaciones CRUD implementadas

Empleado

- Crear: crear_empleado
- Consultar: consultar_empleados
- Actualizar: actualizar_empleado
- Eliminar: eliminar_empleado

Departamento

- Crear: crear_departamento
- Consultar: consultar_departamentos
- Actualizar: actualizar_departamento
- Eliminar: eliminar_departamento

Proyecto

- Crear: crear_proyecto
- Consultar: consultar_proyectos
- Actualizar: actualizar_proyecto
- Eliminar: eliminar_proyecto

RegistroTiempo

- Crear: crear_registro_tiempo
- Consultar: consultar_registros_tiempo
- Actualizar: actualizar_registro_tiempo
- Eliminar: eliminar_registro_tiempo

Cada operación utiliza consultas parametrizadas para evitar concatenar directamente los valores introducidos por el usuario.

7. Manejo de errores y validaciones

La aplicación controla errores mediante excepciones específicas:

- ValueError: datos inválidos, identificadores incorrectos, salarios negativos o fechas mal formadas.
- sqlite3.Error: errores de conexión, restricciones, duplicados o consultas SQL.
- OSError: problemas al exportar archivos.
- TypeError: tipos incompatibles durante conversiones o exportaciones.

La interfaz captura los errores y muestra un mensaje al usuario mediante messagebox, evitando que la ventana se cierre inesperadamente.

También se validan las siguientes reglas:

- Los identificadores deben ser enteros positivos.
- El correo debe contener @.
- Los nombres y estados no pueden estar vacíos.
- El salario y las horas no pueden ser negativos.
- Un registro no puede superar 24 horas.
- Las fechas deben tener formato AAAA-MM-DD.
- Las entidades relacionadas deben existir antes de crear un registro de tiempo.

8. Uso de la interfaz

Desde PowerShell, ubicándose en la carpeta del proyecto, se ejecuta:

```
py interfaz.py
```

La ventana muestra cuatro pestañas:

1. Departamentos: permite crear, actualizar y eliminar departamentos.
2. Empleados: permite registrar empleados, actualizar salarios y eliminarlos.
3. Proyectos: permite crear proyectos, actualizar su estado y eliminarlos.
4. Registro de horas: permite registrar horas, actualizarlas y eliminarlas.

La base de datos se guarda en ecotech.db en la misma carpeta del proyecto.

9. Pruebas realizadas

Se realizaron las siguientes comprobaciones:

- Compilación de ecotech.py e interfaz.py mediante py_compile.
- Importación correcta del módulo de interfaz.
- Ejecución de la demostración CRUD incluida en ecotech.py.

- Creación, consulta, actualización y eliminación de empleados.
- Creación, consulta, actualización y eliminación de departamentos.
- Creación, consulta, actualización y eliminación de proyectos.
- Creación, consulta, actualización y eliminación de registros de tiempo.
- Confirmación de que Usuario es una clase abstracta.
- Confirmación del límite máximo de 24 horas diarias.
- Revisión del editor sin errores detectados en ambos archivos.

La demostración final mostró el mensaje:

Demostracion CRUD completada correctamente.

10. Cumplimiento de los criterios de evaluación

Criterio	Evidencia de cumplimiento
2.1.1.G.1	Las siete clases del UML, sus atributos, métodos, constructores y relaciones están implementados en ecotech.py.
2.1.1.I.2	Los módulos contienen docstrings que explican cómo se traducen las entidades y relaciones UML a Python.
2.1.2.G.3	Se aplican atributos privados, propiedades, herencia, abstracción y reutilización desde Usuario.
2.1.2.I.4	La abstracción de Usuario, el encapsulamiento y la decisión sobre Administrador están documentados.
2.1.3.G.5	sqlite3 crea el esquema y se implementa CRUD completo para empleados, departamentos, proyectos y registros.
2.1.3.I.6	inicializar_bd() documenta la conexión, las claves foráneas y la creación de tablas.
2.1.4.G.7	Se utilizan try/except y excepciones específicas para entradas, SQLite y archivos.
2.1.4.I.8	Los docstrings y el informe explican qué errores se controlan y cómo se protegen los datos.
2.1.5.G.9	La solución está dividida en backend e interfaz, con nombres consistentes y funciones fáciles de probar.
2.1.5.I.10	Al final de ecotech.py se incluye # NOTAS PARA EVALUACIÓN CRÍTICA DE IA.

11. Evaluación crítica y aspectos pendientes

El informe técnico original identificó dos aspectos que requerían revisión manual:

1. Las dependencias de Administrador y ReporteManager no tenían etiquetas ni multiplicidades explícitas en el UML.
2. Administrador no tenía atributos propios.

La implementación conserva ambas decisiones, pero las deja documentadas para que puedan justificarse o modificarse según los requerimientos reales.

También conviene revisar en una siguiente versión:

- Reglas exactas de permisos y autenticación.
- Validación completa de correos y teléfonos.
- Uso de librerías reales para exportar PDF y Excel.
- Pruebas automatizadas con unittest o pytest.
- Separación adicional en módulos como modelos.py, base_datos.py e interfaz.py si el proyecto crece.
- Copias de seguridad y migraciones para una base de datos de producción.

12. Conclusión

La solución implementa el sistema EcoTech Solutions siguiendo el modelo UML entregado y agrega los elementos necesarios para convertirlo en una aplicación funcional: persistencia SQLite, operaciones CRUD, validaciones, manejo de excepciones e interfaz gráfica interactiva.

La separación entre `ecotech.py` e `interfaz.py` resuelve el problema de interacción con el usuario sin mezclar la presentación con la lógica del sistema. El backend puede probarse de forma independiente y la interfaz permite operar los datos mediante formularios y tablas.

La implementación cumple los 10 criterios solicitados y deja identificadas las decisiones que deben ser revisadas manualmente como parte de la evaluación crítica del uso de herramientas de IA.